

# Технологии и методы программирования

## лекция 4

Ищенко Алексей Петрович

# Программа. Процесс обработки данных. Программное средство

- **Цель программирования** – описание процесса обработки данных (ПОД).
- **Данные** – это представление фактов и идей в формализованном виде, пригодном для передачи и переработки в некотором ПОД.
- **Информация** – это смысл, который придаётся данным при их переработке.
- **Обработка данных** – систематическое выполнение некоторой последовательности действий с данными. Данные представляются и хранятся на носителях.
- Совокупность носителей данных, используемых при некоторой обработке данных, называется **информационной средой**.
- Набор данных, содержащихся в какой-либо момент в информационной среде, называют **состоянием этой среды**.

# Программа. Процесс обработки данных. Программное средство

- Следовательно, ПОД можно описать как последовательность сменяющих друг друга состояний некоторой информационной среды. Чтобы описать ПОД, необходимо определить последовательность состояний заданной информационной среды.
- Для того чтобы по заданному описанию ПОД порождался на ЭВМ автоматически, необходимо формализовать это описание. Такое описание называют программой.
- Программа должна быть понятна и человеку, поэтому программы составляются на удобном, формализованном языке программирования.
- Программа переводится на язык соответствующего компьютера с помощью другой программы-транслятора.

# Подготовительная работа перед составлением программы:

- уточнение постановки;
- выбор метода решения;
- выяснение специфики области применения;
- выяснение общей организации программы и т.д.

Результаты подготовительной работы оформляются в виде некоторой документации. Это значительно упрощает понимание человеком программы.

Программное средство (ПС) – программа или логически связанная совокупность программ, размещённая на носителе и снабжённая документацией.

# Документация ПС позволяет понять:

- какие функции выполняет та или иная программа ПС;
- как подготовить исходные данные;
- какие программы содержит ПС;
- как запустить на выполнение;
- что означают полученные результаты;
- как её можно модифицировать.

# Понятие правильности программы

- Считается, что продуктом ТП является ПС, содержащее программы, выполняющие требуемые функции.
- Под программой часто понимают правильную программу, т.е. программу, не содержащую ошибок. Однако понятие ошибки в программе трактуется многими программистами неоднозначно.
- По мнению Майерса в программе имеется ошибка, если она не выполняет того, что разумно ожидать от неё пользователю. При этом разумность определяет документация. Поэтому понятие ошибки не формализовано.
- А так как в программном средстве программа логически связана с программной документацией, правильно говорить не об ошибке в программе, а об ошибке в ПС.
- Частным случаем ошибки может быть несогласованность документации с ПС. Так как задание на ПС неформально, то нельзя доказать правильность программы формальными методами.
- Тестирование не показывает правильность ПС, а лишь иллюстрирует наличие в нём ошибки.

# Надежность программного средства

- Альтернативой правильности является надёжность.
- **Надёжность ПС** – способность ПС безотказно выполнять определённые функции при заданных условиях в течение заданного времени и с достаточно большой вероятностью.
- Под отказом понимают наличие в ПС ошибки. Причём нужно отметить, что надёжное ПС не исключает наличия в нём ошибки. Важно, чтобы при практическом применении ПС эти ошибки проявлялись достаточно редко. Убедиться, что ПС обладает следующими свойствами, можно тестированием или практическим применением. Практически мы разрабатываем лишь надёжные, а не правильные ПС.
- Программные средства обладают различной степенью надёжности. Некоторые ошибки выражают лишь неудобства, а некоторые могут вызывать катастрофические последствия. Поэтому для оценки надёжности иногда используют дополнительный показатель, который учитывает стоимость или вред каждого отказа.

# Проблемы разработки сложных программных систем

- Большинство современных программных систем объективно очень сложны. Эта сложность обуславливается многими причинами, главной из которых является логическая сложность решаемых ими задач.
- Пока вычислительных установок было мало и их возможности были ограничены, ЭВМ применяли в очень узких областях науки и техники, причём, в первую очередь, там, где решаемые задачи были хорошо детерминированы и требовали значительных вычислений. В наше время, когда созданы мощные компьютерные сети, появилась возможность переложить на них решение сложных ресурсоёмких задач, о компьютеризации которых раньше никто и не думал. Сейчас в процесс компьютеризации вовлекаются совершенно новые предметные области, а для уже освоенных областей усложняются сложившиеся постановки задач.

# Проблемы разработки сложных программных систем

- Дополнительными факторами, увеличивающими сложность разработки программных систем, являются:
  - сложность формального определения требований к программным системам;
  - отсутствие удовлетворительных средств описания поведения дискретных систем с большим числом состояний при недетерминированной последовательности входных воздействий;
  - коллективная разработка;
  - необходимость увеличения степени повторяемости кодов.

# Проблемы разработки сложных программных систем

Сложность определения требований к программным системам обусловливается двумя факторами. Во-первых, при определении требований необходимо учесть большое количество различных факторов. Во-вторых, разработчики программных систем не являются специалистами в автоматизируемых предметных областях, а специалисты в предметной области, как правило, не могут сформулировать проблему в нужном ракурсе.

# Проблемы разработки сложных программных систем

- Отсутствие удовлетворительных средств формального описания поведения дискретных систем. В процессе создания программных систем используют языки сравнительно низкого уровня. Это приводит к ранней детализации операций в процессе создания программного обеспечения и увеличивает объём описаний разрабатываемых продуктов, который, как правило, превышает сотни тысяч операторов языка программирования. Средств же, позволяющих детально описывать поведение сложных дискретных систем на более высоком уровне, чем универсальный язык программирования, не существует.

# Проблемы разработки сложных программных систем

- Коллективная разработка. Из-за больших объёмов проектов разработка программного обеспечения ведётся коллективом специалистов. Работая в коллективе, отдельные специалисты должны взаимодействовать друг с другом, обеспечивая целостность проекта, что при отсутствии удовлетворительных средств описания поведения сложных систем, упоминавшемся выше, достаточно сложно. Причём чем больше коллектив разработчиков, тем сложнее организовать процесс работы.

# Проблемы разработки сложных программных систем

- Необходимость увеличения степени повторяемости кодов. На сложность разрабатываемого программного продукта влияет и то, что для увеличения производительности труда компании стремятся к созданию библиотек компонентов, которые можно было бы использовать в дальнейших разработках. Однако в этом случае компоненты приходится делать более универсальными, что в конечном итоге увеличивает сложность разработки.

Вместе взятые, эти факторы существенно увеличивают сложность процесса разработки. Однако очевидно, что все они напрямую связаны со сложностью объекта разработки – программной системы.

# Основные подходы к разработке ПС

1. Разработка носит творческий характер.
2. Особенность разрабатываемого продукта.
3. Продукт разработки (или ПС) при своём использовании не расходуется и не расходует используемых ресурсов.

# Разработка носит творческий характер

- На каждом этапе нужно принимать обдуманные решения, производить какой-либо выбор, а не руководствоваться какой-либо последовательностью механических действий. То есть разработка ПС близка разработке или проектированию сложных устройств. Творческий характер присутствует на всех этапах разработки ПС.

# Особенность разрабатываемого продукта

- ПС представляет собой совокупность некоторых текстов (статических объектов). Смысл же (семантика) этих объектов выражается процессами обработки данных и действиями пользователей по запуску этих процессов (т.е. является динамическим). Это определяет специфический двойственный характер программных средств.

# Блочно-иерархический подход к созданию сложных систем

- Подавляющее большинство сложных систем как в природе, так и в технике имеет иерархическую внутреннюю структуру. Это объясняется тем, что обычно связи элементов сложных систем различны как по типу, так и по силе, что и позволяет рассматривать эти системы как некоторую совокупность взаимозависимых подсистем. Внутренние связи элементов таких подсистем сильнее, чем связи между подсистемами. Например, компьютер состоит из процессора, памяти и внешних устройств, а Солнечная система включает Солнце и планеты, вращающиеся вокруг него.
- В свою очередь, используя то же различие связей, можно каждую подсистему разделить на подсистемы и т.д. до самого нижнего «элементарного» уровня, причём выбор уровня, компоненты которого следует считать элементарными, остаётся за исследователем. На элементарном уровне система, как правило, состоит из немногих типов подсистем, по-разному скомбинированных и организованных. Иерархии такого типа получили название «целое-часть».

# Блочно-иерархический подход к созданию сложных систем

- Поведение системы в целом обычно оказывается сложнее поведения отдельных частей, причём из-за более сильных внутренних связей особенности системы в основном обусловлены отношениями между её частями, а не частями как таковыми.
- В природе существует ещё один вид иерархии – иерархия «простое-сложное», или иерархия развития (усложнения) систем в процессе эволюции. В этой иерархии любая функционирующая система является результатом развития более простой системы. Именно данный вид иерархии реализуется механизмом наследования объектно-ориентированного программирования.

# Блочно-иерархический подход к созданию сложных систем

- Будучи в значительной степени отражением природных и технических систем, программные системы обычно иерархические, т.е. обладают описанными выше свойствами. На этих свойствах иерархических систем строится блочно-иерархический подход к их исследованию или созданию. Этот подход предполагает сначала создавать части таких объектов (блоки, модули), а затем собирать из них сам объект.
- Процесс разбиения сложного объекта на сравнительно независимые части получил название декомпозиции. При декомпозиции учитывают, что связи между отдельными частями должны быть слабее, чем связи элементов внутри частей. Кроме того, чтобы из полученных частей можно было собрать разрабатываемый объект, в процессе декомпозиции необходимо определить все виды связей частей между собой.

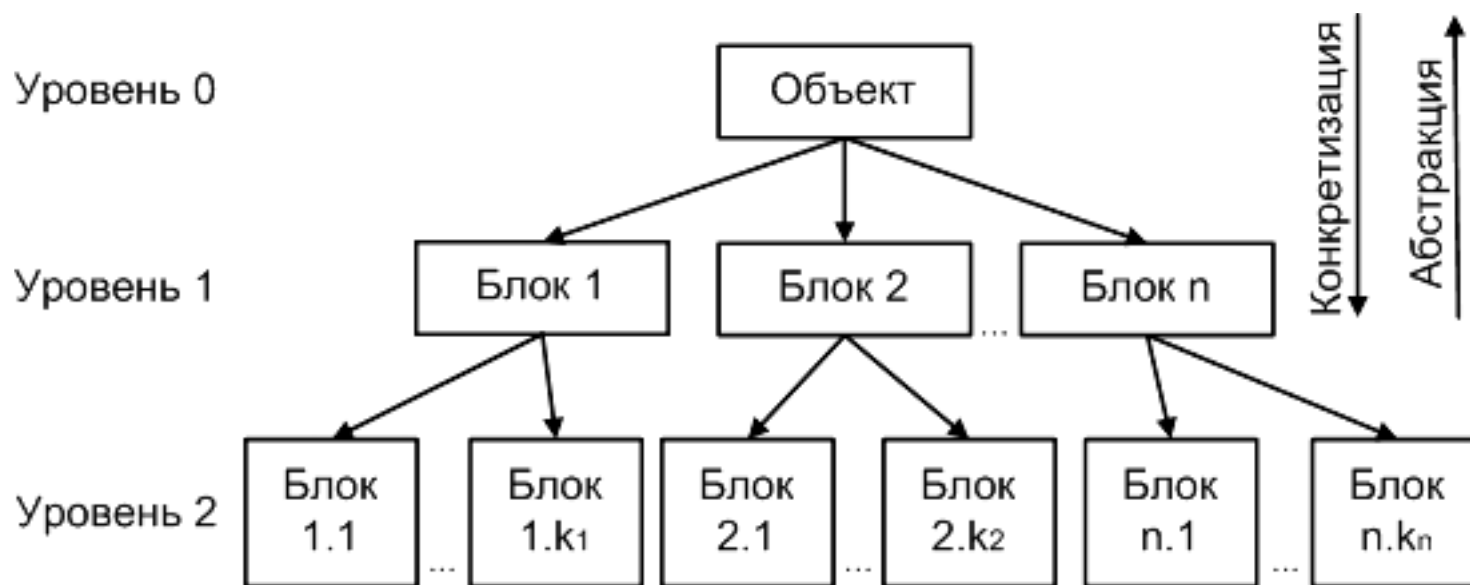
# Блочно-иерархический подход к созданию сложных систем

- При создании очень сложных объектов процесс декомпозиции выполняется многократно: каждый блок, в свою очередь, декомпозируют на части, пока не получают блоки, которые сравнительно легко разработать. Данный метод разработки получил название пошаговой детализации.
- Существенно и то, что в процессе декомпозиции стараются выделить аналогичные блоки, которые можно было бы разрабатывать на общей основе. Таким образом, как упоминалось выше, обеспечивают увеличение степени повторяемости кодов и, соответственно, снижение стоимости разработки.

# Блочно-иерархический подход к созданию сложных систем

- Результат декомпозиции обычно представляют в виде схемы иерархии, на нижнем уровне которой располагают сравнительно простые блоки, а на верхнем – объект, подлежащий разработке. На каждом иерархическом уровне описание блоков выполняют с определённой степенью детализации, абстрагируясь от несущественных деталей. Следовательно, для каждого уровня используют свои формы документации и свои модели, отражающие сущность процессов, выполняемых каждым блоком.
- Так, для объекта в целом, как правило, удаётся сформулировать лишь самые общие требования, а блоки нижнего уровня должны быть специфицированы так, чтобы из них действительно можно было собрать работающий объект. Другими словами, чем больше блок, тем более абстрактным должно быть его описание.

# Соотношение абстрактного и конкретного в описании блоков при блочно-иерархическом подходе



# Блочно-иерархический подход к созданию сложных систем

- При соблюдении этого принципа разработчик сохраняет возможность осмысления проекта и, следовательно, может принимать наиболее правильные решения на каждом этапе, что называют локальной оптимизацией (в отличие от глобальной оптимизации характеристик объектов, которая для действительно сложных объектов не всегда возможна).

# Блочно-иерархический подход к созданию сложных систем

В основе блочно-иерархического подхода лежат декомпозиция и иерархическое упорядочение. Важную роль играют также следующие принципы:

- формализация – строгость методического подхода;
  - повторяемость – необходимость выделения одинаковых блоков для удешевления и ускорения разработки;
  - локальная оптимизация – оптимизация в пределах уровня иерархии;
  - непротиворечивость – контроль согласованности элементов между собой;
  - полнота – контроль на присутствие лишних элементов.
- Совокупность языков моделей, постановок задач, методов описаний некоторого иерархического уровня принято называть уровнем проектирования.

# Блочно-иерархический подход к созданию сложных систем

Каждый объект в процессе проектирования, как правило, приходится рассматривать с нескольких сторон. Различные взгляды на объект проектирования принято называть аспектами проектирования.

Помимо того, что использование блочно-иерархического подхода делает возможным создание сложных систем, он также:

- упрощает проверку работоспособности как системы в целом, так и отдельных блоков;
- обеспечивает возможность модернизации систем, например замены ненадёжных блоков с сохранением их интерфейсов.

Необходимо отметить, что использование блочно-иерархического подхода применительно к программным системам стало возможным только после конкретизации общих положений подхода и внесения некоторых изменений в процесс проектирования.

При этом структурный подход учитывает только свойства иерархии «целое-часть», а объектный – использует ещё и свойства иерархии «простое-сложное».

# Оценка качества процессов создания программного обеспечения

Текущий период на рынке программного обеспечения характеризуется переходом от штучного ремесленного производства программных продуктов к их промышленному созданию. Соответственно, возросли требования к качеству разрабатываемого программного обеспечения, что требует совершенствования процессов их разработки. На настоящий момент существует несколько стандартов, связанных с оценкой качества этих процессов, которое обеспечивает организация- разработчик. К наиболее известным относят:

- международные стандарты серии ISO 9000 (ISO 9000 – ISO 9004);
- CMM (Capability Maturity Model) – модель зрелости (совершенствования) процессов создания программного обеспечения, предложенная SEI (Software Engineering Institute – Институт программирования при университете Карнеги–Меллон);
- рабочая версия международного стандарта ISO/IEC 15504: Information Technology – Software Process Assessment; эта версия более известна под названием SPICE (Software Process Improvement and Capability dEtermination – определение возможностей и улучшение процесса создания программного обеспечения).

# Оценка качества процессов создания программного обеспечения

- Единый стандарт оценки программных процессов SPICE предполагает обеспечение постоянного улучшения процессов разработки программного обеспечения и может быть применен не только к организации в целом, но и к отдельно взятым процессам. Стандарт позволяет проводить оценку проектирования программного обеспечения и при этом выявлять возможности улучшения процесса. В некоторых случаях стандарт имеет преимущество перед группой стандартов ISO 9000, поскольку предоставляет более полный набор средств по обеспечению качества и улучшению процессов.
- Оба направления зарекомендовали себя как достаточно жизнеспособные, но имеющие ряд недостатков. Общие принципы управления процессами, изложенные в требованиях стандартов семейства ISO 9000, дают возможность выбрать наиболее подходящий метод обеспечения и оценки качества процесса проектирования конкретного программного продукта и способствуют развитию новых методов управления и оценки. В то же время SPICE даёт возможность существенно сократить время на «отслеживание» и оценку процессов проектирования за счёт конкретных методик, но не позволяет рассматривать процесс проектирования программного продукта и качество самого продукта как систему.

# Оценка качества процессов создания программного обеспечения

- Существуют и разнообразные методы оценки программных средств, которые должны обеспечивать требования различных групп потребителей. Следует отметить, что в данном случае круг потребителей такого рода продукции несколько расширен, что связано со специфическими особенностями программного обеспечения (ПО), поскольку к «классическим» группам потребителей (государство, организация, конкретный пользователь) необходимо добавить составляющую внешней среды (например, при использовании во время функционирования ПО локальных, глобальных сетей и т.п.) и потребителей, обслуживающих ПО.
- Для определения групп потребительских свойств могут быть использованы как отечественные, так и международные нормативные документы.

# Российские стандарты предлагают использовать следующие группы и комплексные показатели качества:

- показатели надёжности программных средств (ПС) (устойчивость функционирования, работоспособность);
- показатели сопровождения (структурность, простота конструкции, наглядность, повторяемость);
- показатели удобства применения (лёгкость освоения, доступность эксплуатационных программных документов, удобство эксплуатации и обслуживания);
- показатели эффективности (уровень автоматизации, временная эффективность, ресурсоёмкость);
- показатели универсальности (гибкость, мобильность, модифицируемость);
- показатели корректности (полнота реализации, согласованность, логическая корректность, проверенность).

# Согласно ГОСТ Р ИСО 9126 следует обеспечивать следующие качественные характеристики программных средств:

- функциональные возможности (Functionality) – набор атрибутов, относящихся к сути набора функций и их конкретным свойствам. Функциями являются те, которые реализуют установленные или предполагаемые потребности;
- надёжность (Reliability) – набор атрибутов, относящихся к способности программного обеспечения сохранять свой уровень качества функционирования при установленных условиях за установленный период времени;
- практичность (Usability) – набор атрибутов, относящихся к объёму работ, требуемых для использования и индивидуальной оценки такого использования определенным или предполагаемым кругом пользователей;
- эффективность (Efficiencies) – набор атрибутов, относящихся к соотношению между уровнем качества функционирования программного обеспечения и объемом используемых ресурсов при установленных условиях;
- сопровождаемость (Maintainability) – набор атрибутов, относящихся к объёму работ, требуемых для проведения конкретных изменений (модификаций);
- мобильность (Portability) – набор атрибутов, относящихся к способности программного обеспечения быть перенесенным из одного окружения в другое.

# Приёмы обеспечения технологичности программного обеспечения

- В условиях индустриального подхода к разработке и сопровождению программного обеспечения особый вес приобретают технологические характеристики разрабатываемых программ. Для обеспечения необходимых технологических свойств применяют специальные технологические приёмы и следуют определённым методикам, сформулированным всем предыдущим опытом создания программного обеспечения. К таким приёмам и методикам относят правила декомпозиции, методы проектирования, программирования и контроля качества, которые под общим названием «структурный подход к программированию» были сформулированы ещё в 60-х гг. XX в.

# Приёмы обеспечения технологичности программного обеспечения

В его основу были положены следующие основные концепции:

- нисходящая разработка;
- модульное программирование;
- структурное программирование;
- сквозной структурный контроль.

# Приёмы обеспечения технологичности программного обеспечения

- Под технологичностью понимают качество проекта программного продукта, от которого зависят трудовые и материальные затраты на его реализацию и последующие модификации.
- Хороший проект сравнительно быстро и легко кодируется, тестируется, отлаживается и модифицируется.
- Технологичность программного обеспечения определяется проработанностью его моделей, уровнем независимости модулей, стилем программирования и степенью повторного использования кодов.
- Чем лучше проработана модель разрабатываемого программного обеспечения, тем чётче определены подзадачи и структуры данных, хранящие входную, промежуточную и выходную информацию, тем проще их проектирование и реализация и меньше вероятность ошибок, для исправления которых потребуется существенно изменять программу.

# Приёмы обеспечения технологичности программного обеспечения

- Чем выше независимость модулей, тем их легче понять, реализовать, модифицировать, а также находить в них ошибки и исправлять их.
- Стил программирования, под которым понимают стиль оформления программ и их «структурность», также существенно влияет на читаемость программного кода и количество ошибок программирования.
- Увеличение степени повторного использования кодов предполагает как использование ранее разработанных библиотек подпрограмм или классов, так и унификацию кодов текущей разработки. Причём для данного критерия ситуация не так однозначна, как в предыдущих случаях: если степень повторного использования кодов повышается искусственно (например, путём разработки «суперуниверсальных» процедур), то технологичность проекта может существенно снизиться.
- Как следует из определения, высокая технологичность проекта особенно важна, если разрабатывается программный продукт, рассчитанный на многолетнее интенсивное использование, или необходимо обеспечить повышенные требования к его качеству.

# Приёмы обеспечения технологичности программного обеспечения

- При проектировании достаточно сложного программного обеспечения после определения его общей структуры выполняют декомпозицию компонентов в соответствии с выбранным подходом до получения элементов, которые, по мнению проектировщика, в дальнейшей декомпозиции не нуждаются.

Как упоминалось раньше, в настоящее время используют два способа декомпозиции разрабатываемого программного обеспечения, связанных с соответствующим подходом:

- процедурный (или структурный – по названию подхода);
- объектный.

# Два способа декомпозиции

- Результатом процедурной декомпозиции является иерархия подпрограмм (процедур), в которой функции, связанные с принятием решения, реализуются подпрограммами верхних уровней, а непосредственно обработка – подпрограммами нижних уровней. Это согласуется с принципом вертикального управления, который был сформулирован вместе с другими рекомендациями структурного подхода к программированию. Он также ограничивает возможные варианты передачи управления, требуя, чтобы любая подпрограмма возвращала управление той подпрограмме, которая её вызвала.
- Результатом объектной декомпозиции является совокупность объектов, которые затем реализуют как переменные некоторых специально разрабатываемых типов (классов), представляющих собой совокупность полей данных и методов, работающих с этими полями.

# Модули

- Модулем называют автономно компилируемую программную единицу. Термин «модуль» традиционно используется в двух смыслах. Первоначально, когда размер программ был сравнительно невелик и все подпрограммы компилировались отдельно, под модулем понималась подпрограмма, т.е. последовательность связанных фрагментов программы, обращение к которой выполняется по имени. Со временем, когда размер программ значительно вырос и появилась возможность создавать библиотеки ресурсов: констант, переменных, описаний типов, классов и подпрограмм, термин «модуль» стал использоваться и в смысле автономно компилируемого набора программных ресурсов. Данные модуль может получать и/или возвращать через общие области памяти или параметры.

# Модули

Первоначально к модулям (ещё понимаемым как подпрограммы) предъявлялись следующие требования:

- отдельная компиляция;
- одна точка входа;
- одна точка выхода;
- соответствие принципу вертикального управления;
- возможность вызова других модулей;
- небольшой размер (до 50 – 60 операторов языка);
- независимость от истории вызовов;
- выполнение одной функции.

# Модули

- Требования одной точки входа, одной точки выхода, независимости от истории вызовов и соответствия принципу вертикального управления были вызваны тем, что в то время из-за серьёзных ограничений на объём оперативной памяти программисты были вынуждены разрабатывать программы с максимально возможной повторяемостью кодов. В результате подпрограммы, имеющие несколько точек входа и выхода, не только были обычным явлением, но и считались высоким классом программирования. Следствием же было то, что программы было очень сложно не только модифицировать, но и понять, а иногда и просто полностью отладить.

# Модули

Со временем, когда основные требования структурного подхода стали поддерживаться языками программирования и под модулем стали понимать отдельно компилируемую библиотеку ресурсов, требование независимости модулей стало основным.

Практика показала, что чем выше степень независимости модулей, тем:

- легче разобраться в отдельном модуле и всей программе и, соответственно, тестировать, отлаживать и модифицировать её;
- меньше вероятность появления новых ошибок при исправлении старых или внесении изменений в программу, т.е. вероятность появления «волнового» эффекта;
- проще организовать разработку программного обеспечения группой программистов и легче его сопровождать.

Таким образом, уменьшение зависимости модулей улучшает технологичность проекта. Степень независимости модулей (как подпрограмм, так и библиотек) оценивают двумя критериями: сцеплением и связностью.

**Спасибо за внимание!**

Далее «Сцепление модулей и др...»

Ищенко Алексей Петрович